Szakmai Gyakorlat és FFT Projekt

Színjátszó Tábor Weboldala

(Csalamádé Next.js Projekt) Átfogó Dokumentáció

Készítette:
Nagy Csaba

Számítástechnika IV.

2026

Tartalomjegyzék

Tartalomjegyzék **1**

1. Bevezetés **2**

2. Felhasználói Felületek Tervezése (FFT) **3**
2.1. Célközönség és Personák 3
2.2. Piackutatás és Felhasználói Vélemények 3
2.3. Vizuális Koncepció (Design Tokenek) 4

3. Frontend Architektúra és Keretrendszerek **5**
3.1. Miért a Next.js? 5
3.2. A Root Layout Felépítése (layout.tsx) 5

4. Állapotkezelés: TanStack Query **7**
4.1. A QueryProvider Működése 7

5. Dinamikus Komponensek és Reszponzivitás **9**
5.1. A probléma: Mobilos navigáció és a "Two-Tap" logika 9
5.2. A Reszponzív Állapotgép (State Machine) 10

6. UI/UX Implementáció és Animációk (GSAP) **11**
6.1. Tailwind CSS a gyorsaságért 11
6.2. GSAP és Mikro-interakciók 11

7. Összegzés és Szoftvermérnöki Tanulságok **12**
7.1. Tanulságok és Kihívások 12
7.2. További fejlesztési lehetőségek (Future Work) 12

1. fejezet

Bevezetés

A modern webfejlesztés rohamos fejlődésével a statikus, tisztán információközlésre fókuszáló weboldalak ideje lejárt, különösen akkor, ha a célközönség a vizuális ingerekhez szokott Z és Alfa generáció. A jelen dokumentáció a "Csalamádé" néven ismert diákszínjátszó tábor interaktív weboldalának felépítését tárgyalja, a kezdeti tervezési fázisoktól (FFT) a komplex szoftvermérnöki architektúra felépítéséig.

A dokumentáció részletes technikai elemzést nyújt a projekt forráskódjáról, kitérve arra, hogy miért éppen a **Next.js**, a **TanStack Query** és a **Tailwind CSS** technológiákat választottuk, és hogyan valósítottuk meg a komplex, mobilon is kiválóan teljesítő komponenseket.

2. fejezet

Felhasználói Felületek Tervezése (FFT)

A fejlesztés nulladik lépése a potenciális látogatók azonosítása és a felhasználói élmény (UX) megtervezése volt.

2.1. Célközönség és Personák

A weboldal kettős célközönséggel rendelkezik, amelyeket a következő personák reprezentálnak:

- **Persona 1: A táborozó diák (16-18 éves):** Olyan vizuális ingerekre vágyik, amelyek megmutatják, milyen lesz a tábor hangulata, illetve kik lesznek ott. Egy unalmas, elavult kinézetű oldal elriasztja. Számára az élmény és a videós tartalom a legfontosabb.
- **Persona 2: Nagy Gábor (44 éves, szülő):** Olyan biztonságos, fejlesztő környezetet keres a lányának, ahol értékteremtő munka folyik szakemberek vezetésével. Fájdalompontja, hogy sok tábornak nincs hivatalos weboldala, átláthatatlanok a szervezők, és zavarja a bonyolult, papíralapú jelentkezési formaság.

2.2. Piackutatás és Felhasználói Vélemények

A tervezést egy átfogó piackutatás előzte meg, mely során hasonló táborok online jelenlétét vizsgáltuk:

- **Hagyományos tábori oldalak (pl. WordPress):** Előnyük, hogy az információ pontos, de hátrányuk a statikus, szövegközpontú elrendezés és a lassú, elavult mobilnézet, amely nem rezonál a fiatalabb generációval.

- **Kizárólag Facebook Page használata:** Jó az elérés, de az információk elvesznek a hírfolyamban. Nincs dedikált jelentkezési felület és nehezen kereshetők vissza az archív anyagok.

Kérdőíves felméréseink és mélyinterjúk során kiderült, hogy a diákok 80%-a egy-egy hangulatvideó alapján dönti el, érdekli-e a rendezvény. Ezek alapján született meg a mi **új, kiterjesztett ötletünk**: Információközlés helyett *immerzív élményt* nyújtunk dinamikus hátterekkel, interaktív videós kártyákkal és mozgásközpontú animációkkal.

2.3. Vizuális Koncepció (Design Tokenek)

A Figma tervek alapján a végső koncepció a következő vizuális irányelveket követi:

- **Színek és Téma:** A sötét mód dominál. Ez szükséges a "Pop-out" hatás eléréséhez, így a színes videó-bélyegképek kitűnnek a háttérből.
- **Tipográfia:** A *Museo* betűcsalád (Museo300 és Museo700) használata, amely kerekded, mégis komoly karaktere miatt tökéletesen passzol a játékos, színházi környezethez.

3. fejezet

Frontend Architektúra és Keretrendszerek

A vizuális tervek elkészülte után a legfontosabb döntés a megfelelő technológiai stack kiválasztása volt. A választás a **Next.js** App Router alapú architektúrájára esett.

3.1. Miért a Next.js?

A klasszikus React SPA (Single Page Application) legnagyobb hátránya a gyenge keresőoptimalizálás (SEO) és a lassú kezdeti oldalbetöltés, mivel a böngészőnek le kell töltenie a teljes JavaScript bundle-t a renderelés megkezdése előtt. A Next.js **SSR (Server-Side Rendering)** képessége megoldja ezt a problémát: a szerver már a kész HTML-t küldi a kliensnek, így az oldal azonnal megjelenik, és a keresőrobotok (Google) is tökéletesen tudják indexelni az eseményt.

3.2. A Root Layout Felépítése (layout.tsx)

Minden Next.js (App Router) projekt lelke a `layout.tsx`. Itt konfiguráljuk a globális szolgáltatókat (Providereket) és töltjük be a betűtípusokat a legoptimálisabb módon a `next/font` API használatával.

```
import { LanguageProvider } from "@/context/LanguageContext";
import QueryProvider from "@/components/QueryProvider";
import localFont from "next/font/local";

// Helyi betutipusok betoltese es CSS valtozokkent valo regisztralasa
const museo300 = localFont({
  src: "../public/fonts/Museo300.otf",
  variable: "--font-museo-300",
  display: "swap", // Optimalizalt fontmegjelenites
});
```

```

export default function RootLayout({ children }: { children: React.
    ReactNode }) {
  return (
    <html lang="hu" suppressHydrationWarning>
      <body className={`${museo300.variable} antialiased`}>
        {/* Adatkezelesi reteg */}
        <QueryProvider>
          {/* Nyelvi kontextus */}
          <LanguageProvider>
            <main>{children}</main>
          </LanguageProvider>
        </QueryProvider>
      </body>
    </html>
  );
}
```

Listing 3.1. A projekt alapját képező app/layout.tsx részlete

Mit csinál a fenti kód? 1. **`localFont`**: Beolvassa a lokális *Museo* fontfájlokat, optimalizálja azokat, és egyedi CSS változóként (`-font-museo-300`) adja át a DOM-nak. Ez megakadályozza a CLS (Cumulative Layout Shift) jelenséget, ami drasztikusan javítja a Google Lighthouse pontszámot. 2. **`QueryProvider` & `LanguageProvider`**: Ezek az úgynevezett Context Providerek. A Next.js 13+ App Routerében alapértelmezetten minden komponens Server Component (szerveren fut le). Mivel a nyelvi váltás és az adatlekérdezés gyorsítótára (cache) kliens-oldali interaktivitást igényel, ezekbe a providerekbe csomagoljuk az alkalmazást.

4. fejezet

Állapotkezelés: TanStack Query

Egy modern webalkalmazás szinte sosincs elszigetelve; adatokat kell lekérnie API-kon keresztül (pl. videók adatai, résztvevők listája). Ezt a klasszikus Reactban `useEffect` és `useState` segítségével szokás megoldani, amely azonban rendkívül sebezhető: nincs beépített cache, nincs automatikus újrapróbálkozás hiba esetén, és a komponens újrarajzolásakor felesleges hálózati kérések indulnak.

A projektben erre a **TanStack Query** (korábban React Query) technológiát használtuk.

4.1. A QueryProvider Működése

```
"use client";

import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { useState } from 'react';

export default function QueryProvider({ children }: { children: React.ReactNode }) {
  const [queryClient] = useState(
    () =>
      new QueryClient({
        defaultOptions: {
          queries: {
            staleTime: 60 * 1000, // 1 percig frissnek tekinti az adatot
            retry: 1, // Hiba eseten egyszer probalkozik ujra
          },
        },
      })
  );

```

```
  return (
    <QueryClientProvider client={queryClient}>
      {children}
    </QueryClientProvider>
  );
}
```

Listing 4.1. A QueryProvider.tsx implementációja

Ebben a kódrészletben a `staleTime: 60 * 1000` beállítás garantálja, hogy ha a felhasználó navigál az oldalon és visszatér egy olyan szekcióhoz (pl. a Táborok listájához), az alkalmazás nem küld új API kérést, hanem azonnal, 0 milliszekundum alatt kiszolgálja a gyorsítótárból a memóriában lévő adatot. Ezzel hatalmas szerverterhelést spórolunk meg, és a felhasználói élmény hihetetlenül fluid marad.

5. fejezet

Dinamikus Komponensek és Reszponzivitás

A projekt legkomplexebb és mérnökileg legnagyobb kihívást jelentő része a `CampsSection.tsx` volt. Ennek a szekciónak a feladata a korábbi évek táborainak (videók) megjelenítése egy interaktív, vízszintesen görgethető (Carousel) formátumban.

5.1. A probléma: Mobilos navigáció és a "Two-Tap" logika

Amikor a kártyák vízszintesen vannak felsorakoztatva, a felhasználó ujjal (swipe) próbálja görgetni azokat okostelefonon. A fejlesztés korai szakaszában a swipe mozdulat véletlenül rákattintott egy kártyára, ami váratlanul átirányította a felhasználót a YouTube-ra. Ez rendkívül frusztráló UX-et eredményezett.

A mérnöki megoldás: A "Two-Tap" (dupla koppintásos) logika implementálása aszinkron fókuszkövetéssel.

```
useEffect(() => {
    if (!config.isPortrait) return; // Csak mobilon fut le

    const timeoutId = setTimeout(() => {
        const observer = new IntersectionObserver(
            (entries) => {
                entries.forEach((entry) => {
                    if (entry.isIntersecting) {
                        const cardId = parseInt(entry.target.
   getAttribute('data-card-id') || '1');
                        setCenteredCardId((prev) => {
                            if (prev !== cardId) {
                                setTappedCardId(null); // Uj focusnal
   elveszitjuk a kijelolest
```

```
                                return cardId;
                            }
                            return prev;
                        });
                    }
                });
            },
            {
                root: scrollContainerRef.current,
                rootMargin: '0px -45% 0px -45%', // Pontosan a kepernyo kozepet vizsgalja
                threshold: 0.1,
            }
        );

        cardRefs.current.forEach((element) => {
            if (element) observer.observe(element);
        });

        return () => observer.disconnect(); // Takaritas
    }, 100);

    return () => clearTimeout(timeoutId);
}, [config.isPortrait]);
```

Listing 5.1. Részlet a CampsSection.tsx kódból (Intersection Observer)

Mit csinál a kód? Ahelyett, hogy drága (számításigényes) görgetési eseményeket (`onScroll`) figyelnénk, a böngésző natív, rendkívül hardvergyorsított **Intersection Observer API**-ját használjuk. A `rootMargin: '0px -45% 0px -45%'` beállítás gyakorlatilag egy apró, láthatatlan vonalat hoz létre a képernyő közepén. Amikor a felhasználó görget egy kártyát, és az metszi ezt a középső vonalat, a kártya bekerül a `centeredCardId` állapotba (state). Ekkor színes lesz és megnő. A *második* tudatos érintésre (`isTapped`) pedig elindul a külső videós link. Ezzel a frusztráló kattintási hibákat 100%-ban eliminált uk.

5.2. A Reszponzív Állapotgép (State Machine)

A komponens folyamatosan figyeli a képernyőméretet a `performResize` függvény segítségével (Debouncing technikával, hogy ne terhelje a processzort átméretezéskor). Mobilon dinamikusan változtatja a kártyák méretét (`vw` és `vh` egységek használatával), míg asztali gépen egy mágneses (hover) interakciót tesz lehetővé.

6. fejezet

UI/UX Implementáció és Animációk (GSAP)

Az oldal dizájnja szándékosan szakít a statikus sablonokkal. A vizuális visszajelzések és a fluiditás a márka (Csalamádé) alapértéke.

6.1. Tailwind CSS a gyorsaságért

A komponensek megírásánál a Tailwind CSS-t használtuk. Egy klasszikus BEM architektúrájú CSS helyett a utility-first osztályok (pl. `flex flex-col items-center justify-center pt-0 -mt-30 px-0 pb-10 relative z-10`) lehetővé tették, hogy a stílus a komponens logikájával egy fájlban (*Colocation*) maradjon. Ez javítja a fejlesztési élményt és csökkenti a halott CSS kód mennyiségét, hiszen a fordító (compiler) csak a ténylegesen használt osztályokat építi be a végső CSS fájlba.

6.2. GSAP és Mikro-interakciók

A React beépített animációs könyvtárai (pl. Framer Motion) helyett a projekt a **GSAP (GreenSock Animation Platform)** technológiára épít a nagyobb teljesítmény és kompatibilitás érdekében. A GSAP képes közvetlenül a böngésző GPU-jára küldeni a transzformációs utasításokat. A `/jelentkezz` végponton futó SVG maszkolás, ahol a tábor logója "kitágul", és a felhasználó fizikailag beesik az űrlapba, egy klasszikus GSAP *Timeline* segítségével készült, megszakítva a hagyományos "kattintok-és-betölt-egy-új-oldal" monotóniáját.

7. fejezet

Összegzés és Szoftvermérnöki Tanulságok

A Csalamádé Színjátszó Tábor weboldalának fejlesztése kiváló lehetőséget biztosított arra, hogy a szoftverfejlesztési és objektumorientált elveket egy modern, piacképes termékben integráljuk.

7.1. Tanulságok és Kihívások

A legnagyobb kihívást az animációk, a teljesítmény és az akadálymentesség egyensúlyának megtartása jelentette. Míg az asztali böngészők hardveres gyorsítással könnyen kezelik a GSAP animációkat és a nagyfelbontású `next/image` képeket, addig az olcsóbb, régebbi okostelefonokon agresszív memória-optimalizációra (WebP formátum, csökkentett polyfill) volt szükség a zökkenőmentes görgetés érdekében. Az iOS Safari különleges DOM-kezelési sajátosságai (például a z-index hibák a 3D transzformációknál) számos egyedi fallback írását tették szükségessé.

7.2. További fejlesztési lehetőségek (Future Work)

Bár a jelenlegi verzió rendkívül robusztus és esztétikus, a rendszer jövőbeni skálázhatósága az alábbi modulokkal bővülhet:

1. **Integrált Jelentkezési Backend:** A külső Google Forms leváltása, egy saját Next.js API Route alapú űrlapkezelőre, amely közvetlenül egy relációs adatbázisba (pl. PostgreSQL) dolgozik.

2. **Headless CMS Integráció:** Egy tartalomkezelő rendszer (pl. Strapi vagy Sanity) bekötése, hogy a tábor szervezői programozói tudás nélkül tudják cserélni a videókat és az idézeteket.

Összességében a projekt maradéktalanul teljesítette a célkitűzéseket: a hagyományos, statikus információmegosztást egy dinamikus, vizuálisan magával ragadó webes élménnyé transzformálta.